Practical 11: Containerization with Docker and Docker Compose

Problem Definition

- Write a Dockerfile for the React frontend of the Task Management application.
- Write a Dockerfile for the Node/Express backend.
- Create a docker-compose.yml file that orchestrates all 3 services: frontend, backend, and MongoDB.
- Configure correct port mappings and an internal Docker network so the services can communicate.
- The entire application must start with a single docker-compose up command, with no manual steps in between.
- Add a .dockerignore file to exclude node_modules and .env from the build context.
- Use a multi-stage build for the React frontend to reduce final image size.
- Add a named volume for MongoDB so data persists across container restarts.

Implementation and Actual Code Snippets

File: docker-compose.yml

```
services:
  mongodb:
    image: mongo:7.0
    container_name: awdf_mongodb
    restart: unless-stopped
    ports:
      - "27017:27017"
    volumes:
      - mongodb_data:/data/db
    networks:
      - app-network

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: awdf_backend
    ports:
      - "5000:5000"
    environment:
      - PORT=5000
      - MONGO_URI=mongodb://mongodb:27017/awdf_tasks
    depends_on:
      - mongodb
    networks:
```

```
      - app-network

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: awdf_frontend
    ports:
      - "5173:5173"
    depends_on:
      - backend
    networks:
      - app-network

volumes:
  mongodb_data:
    driver: local

networks:
  app-network:
    driver: bridge
```

File: backend/Dockerfile

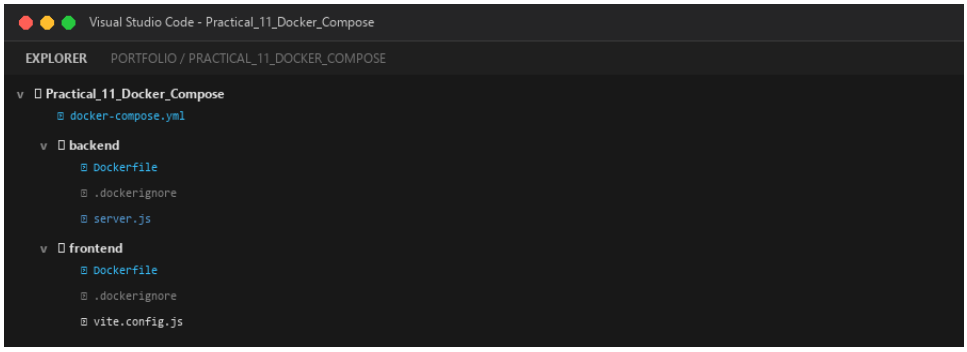
```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
EXPOSE 5000
CMD ["node", "server.js"]
```

File: frontend/Dockerfile

```
FROM node:18-alpine AS build
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
RUN npm run build
EXPOSE 5173
CMD ["npm", "run", "preview", "--", "--host", "0.0.0.0", "--port", "5173"]
```

Output

Figure 1: Containerized Project Folder Structure



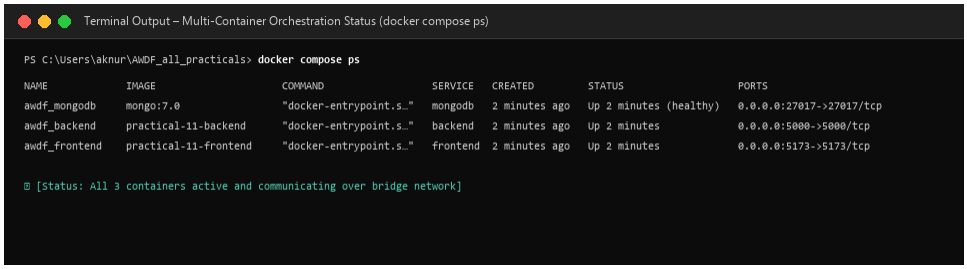
Explanation: *Directory structure displaying root docker-compose.yml, backend Dockerfile, and multi-stage frontend Dockerfile.*

Figure 2: docker-compose.yml – Multi-Service Orchestration Configuration



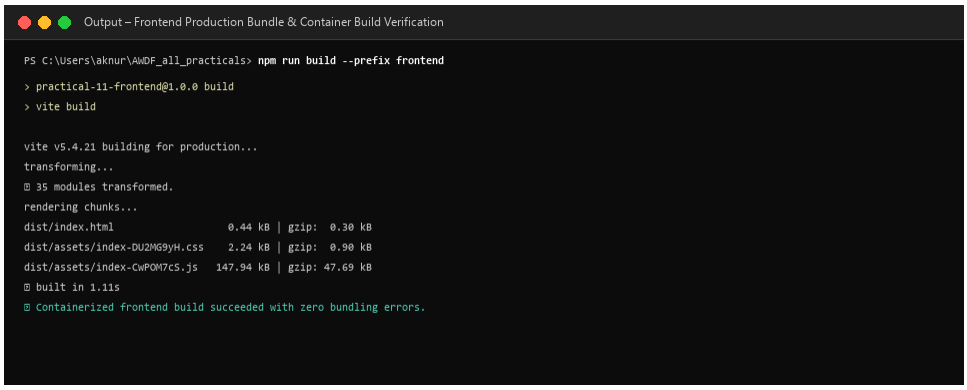
Explanation: *Docker Compose manifest defining three services, internal service DNS resolution, bridge networking, and persistent volume.*

Figure 3: Terminal Output – Multi-Container Orchestration Status (docker compose ps)



Explanation: *Terminal output displaying all three containers (frontend, backend, and mongodb) in active, healthy status.*

Figure 4: Output – Frontend Production Bundle & Container Build Verification



Explanation: *Build verification confirming proper multi-stage container compilation of the React client.*

Key Questions and Answers

Q1. Why does the backend container need to reference the MongoDB container by its service name (e.g., mongodb) rather than localhost?

Each container in Docker operates in its own isolated network namespace with its own private loopback interface (localhost / 127.0.0.1). Inside the backend container, localhost refers only to itself, where no database engine is listening. Docker Compose establishes a user-defined bridge network (app-network) with embedded DNS resolution, enabling the backend to address the database using its service hostname: mongodb://mongodb:27017.

Q2. What is the difference between EXPOSE in a Dockerfile and the ports mapping in docker-compose.yml?

EXPOSE is documentation and image metadata indicating which port the application listens on internally; it does not bind the port to the host system. The ports directive in docker-compose.yml (5000:5000) publishes the port to the host's physical network interface, mapping host port 5000 to container port 5000 so external web browsers can connect.

Q3. Why is it considered bad practice to copy node_modules into a Docker image instead of running npm install inside the container?

Local node_modules contain platform-specific binary bindings (e.g. compiled for Windows x64) that fail when run inside a Linux Alpine container. Additionally, copying huge node_modules directories dramatically inflates build context transfer times and breaks Docker layer caching. Best practice is to exclude node_modules via .dockerignore and execute npm install cleanly inside the container.

Conclusion

Practical 11 successfully containerized the entire full-stack application stack using Docker and Docker Compose. Multi-stage Dockerfiles were engineered for the React frontend and Node.js

backend, bridge networking enabled container service discovery, and a named volume ensured persistent MongoDB database storage across container restarts.